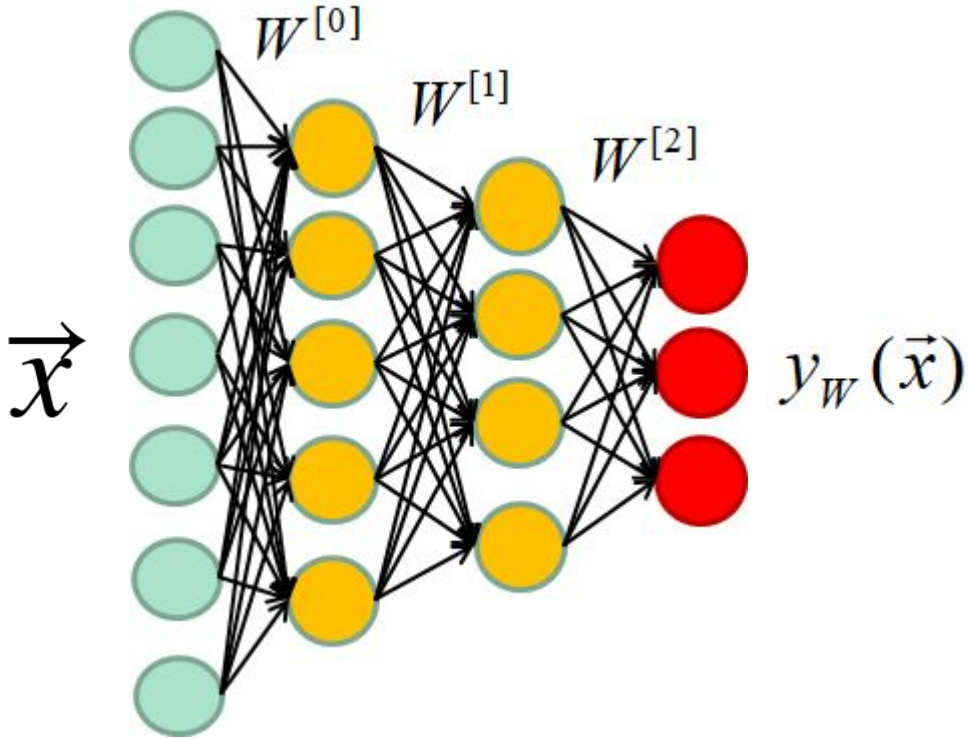
Deep learning for medical imaging school

Hands-on session

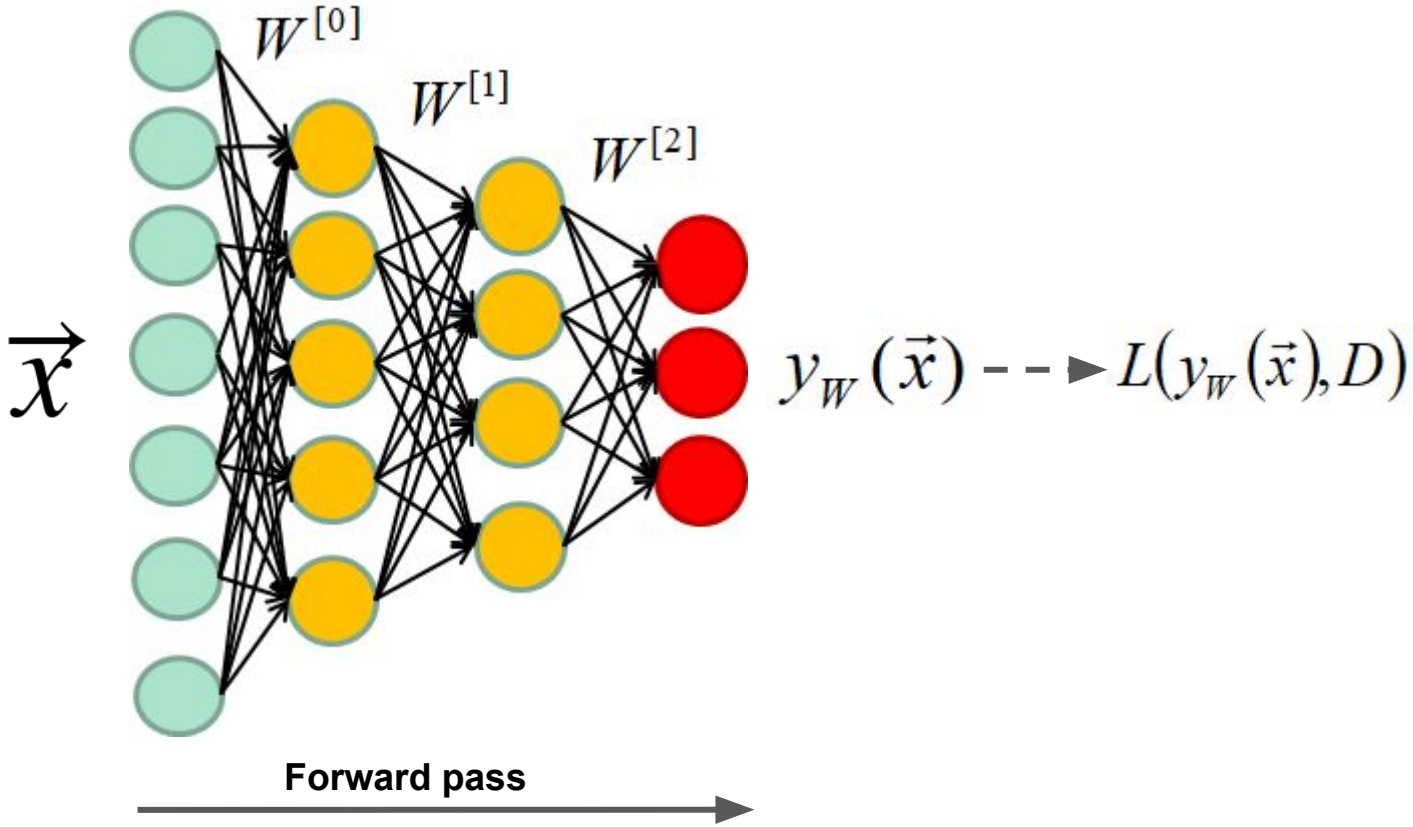
Autoencoders

By

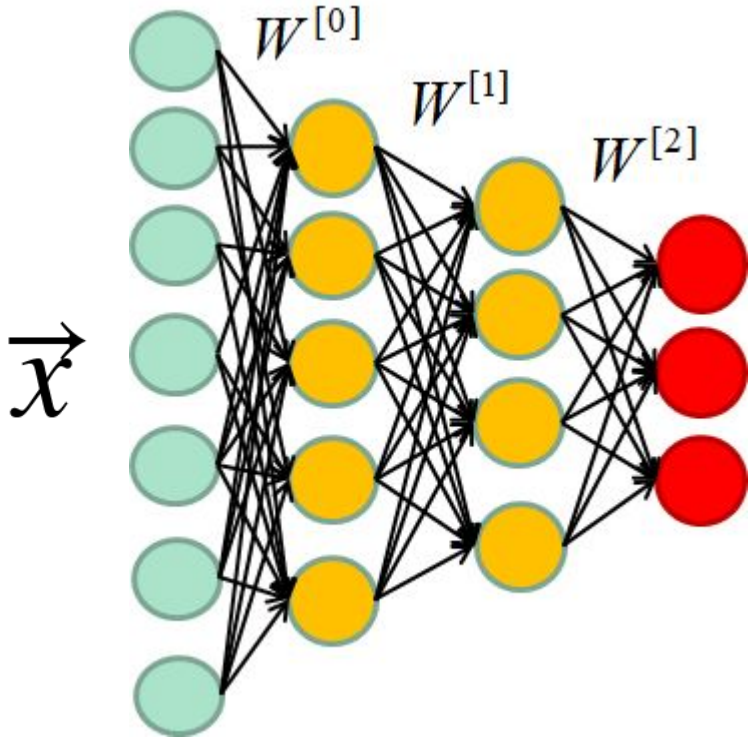
Pierre-Marc Jodoin, Thierry Judge


$W^{[0]}$
$W^{[1]}$
$W^{[2]}$
$\vec{x}$
$y_W(\vec{x})$

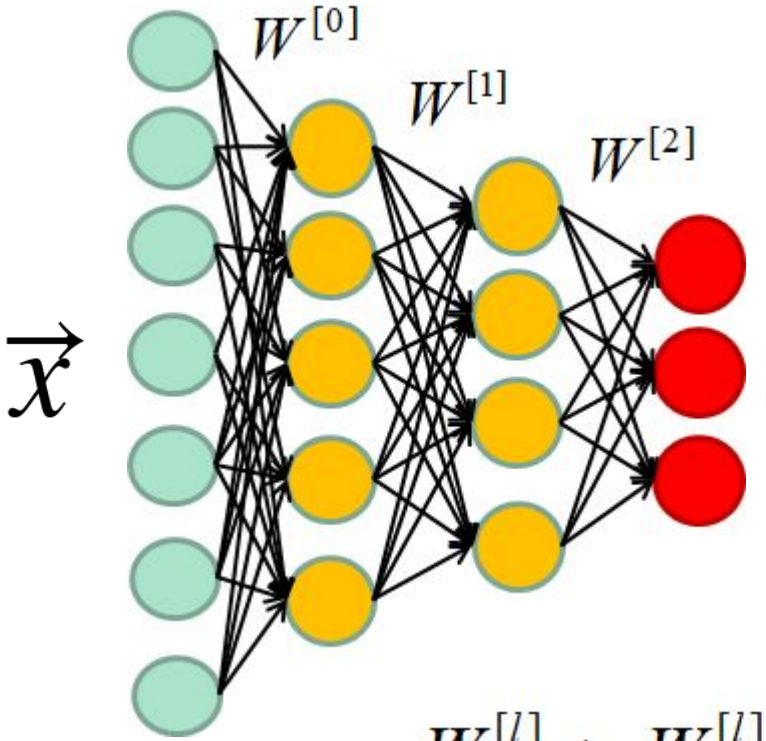
Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \ldots, (\vec{x}_N, t_N)\}$



Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \ldots, (\vec{x}_N, t_N)\}$

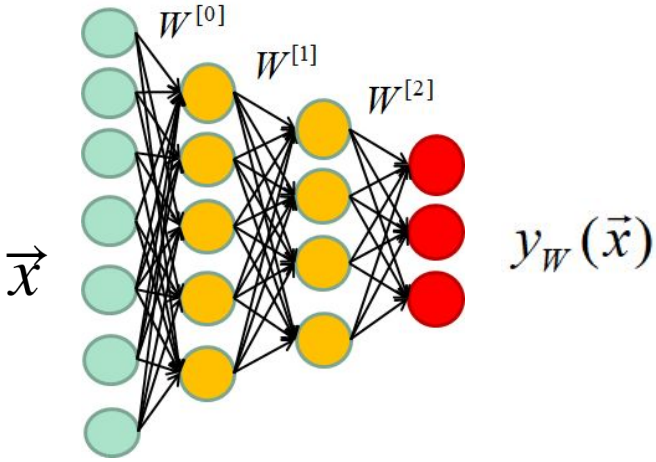


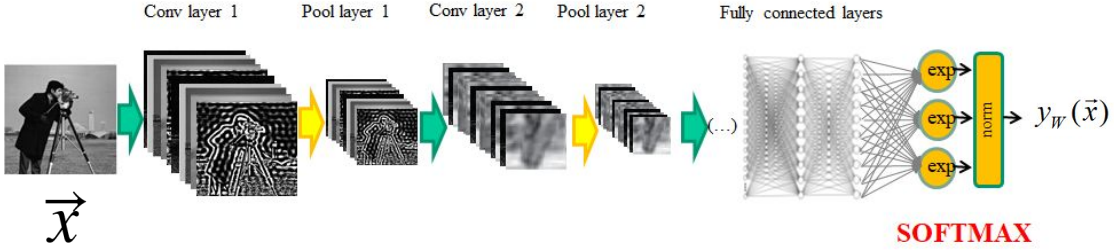
Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \ldots, (\vec{x}_N, t_N)\}$

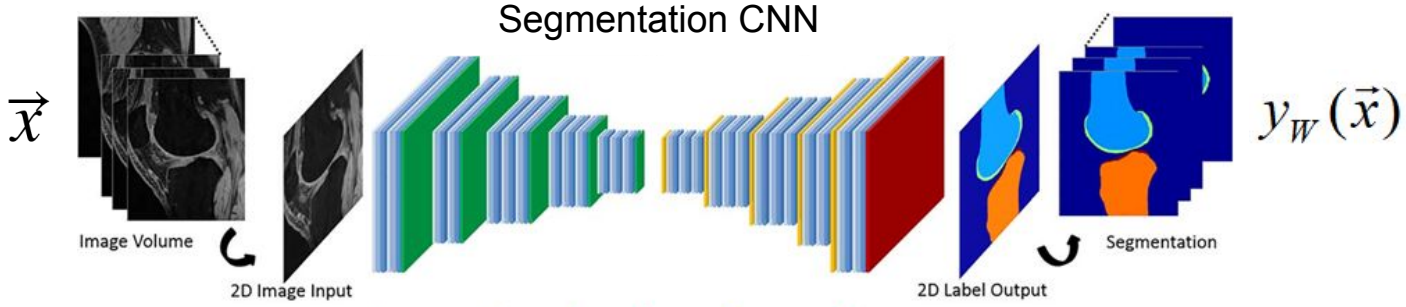


Gradient descent

$$W_j^{[l]} \leftarrow W_j^{[l]} - \eta \frac{\partial\left(L\left(y_W(\vec{x})\right)\right)}{\partial W_j^{[l]}}$$



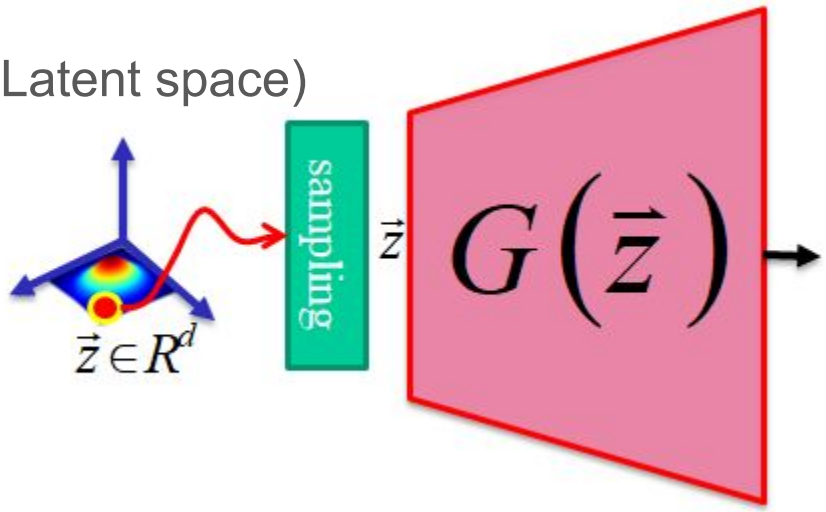




Fang Liu, etal., Deep convolutional neural network and 3D deformable approach for tissue segmentation in musculoskeletal magnetic resonance imaging. in Magnetic resonance in medicine 2018 DOI:10.1002/mrm.26841

GAN

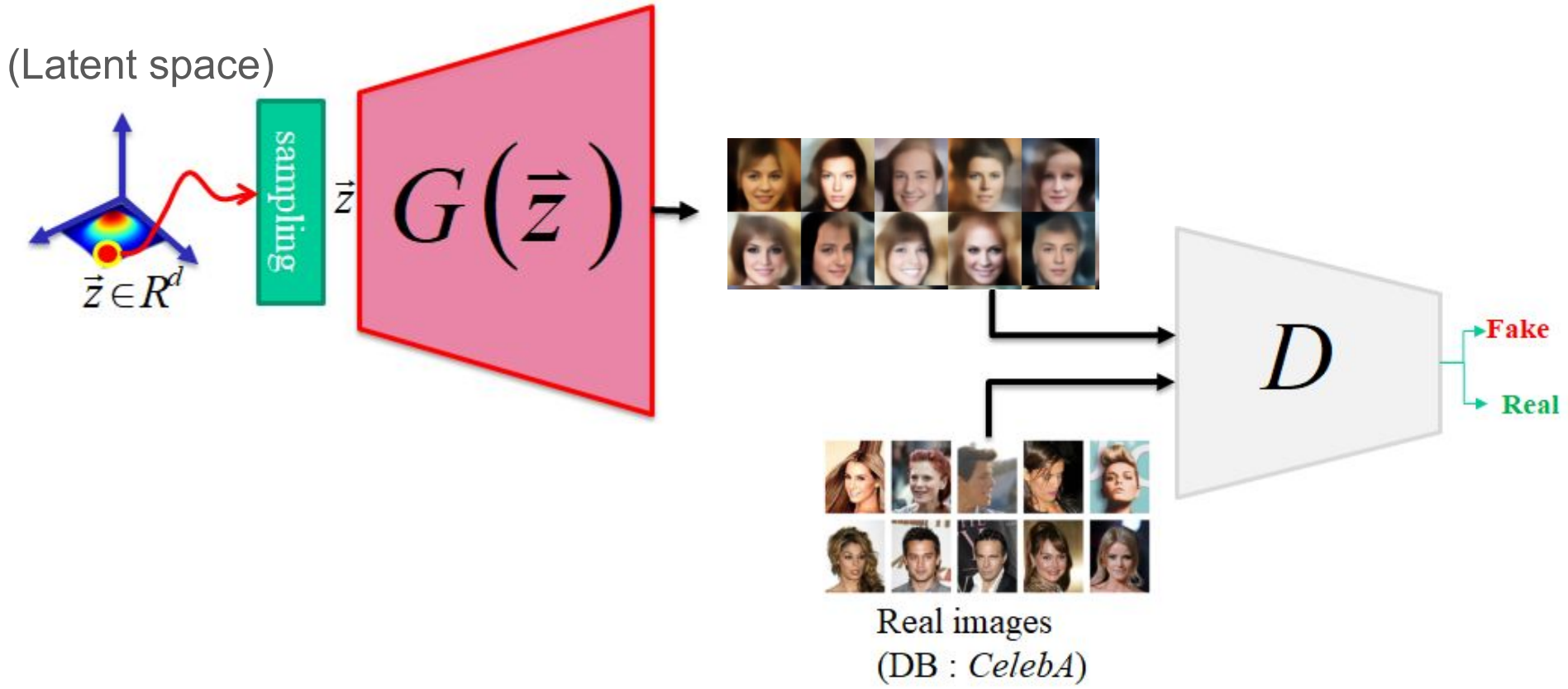
(Latent space)



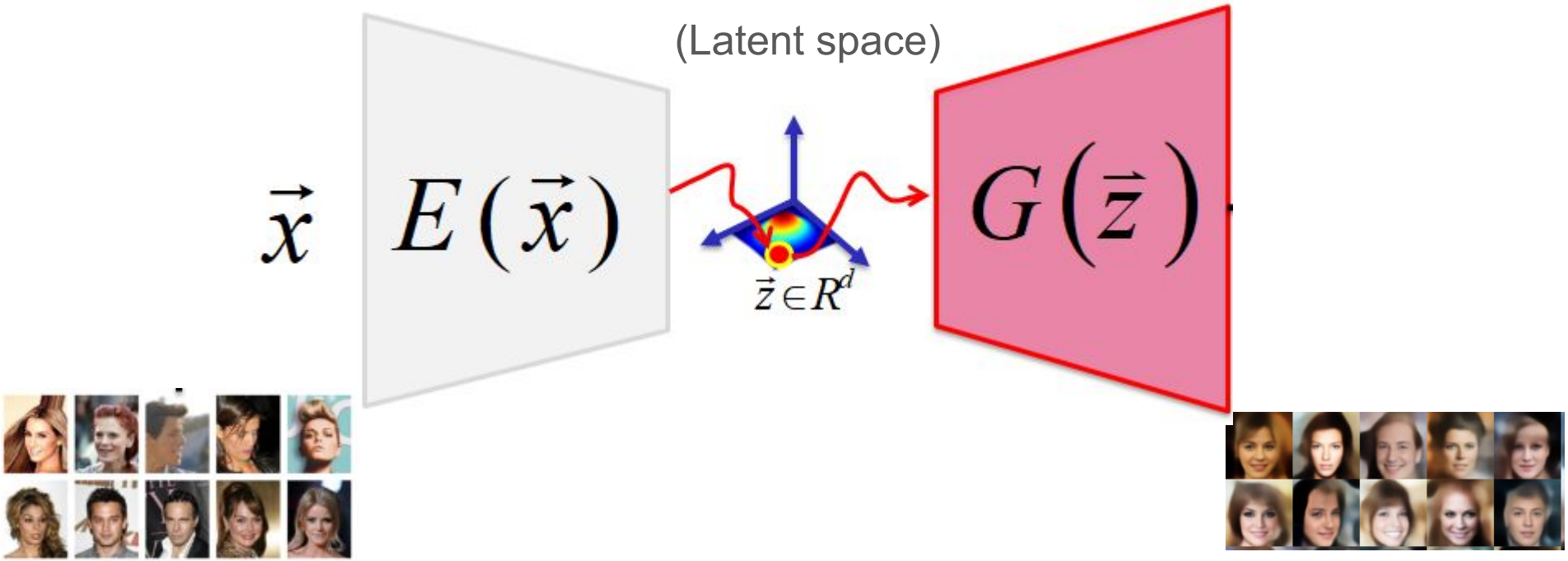
Loss?

Annotated dataset?

GAN

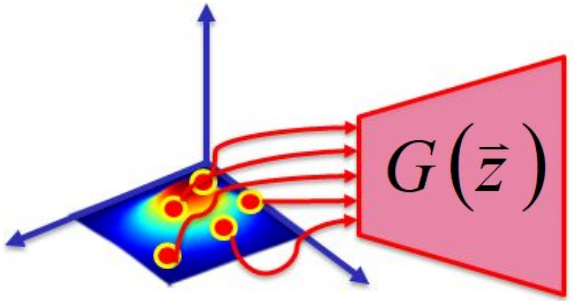

(Latent space)
$\vec{z} \in R^d$
sampling
$\vec{z}$
$G(\vec{z})$
D
Fake
Real
Real images
(DB : *CelebA*)

Autoencoders



Autoencoders (once training is over)

(Latent space)



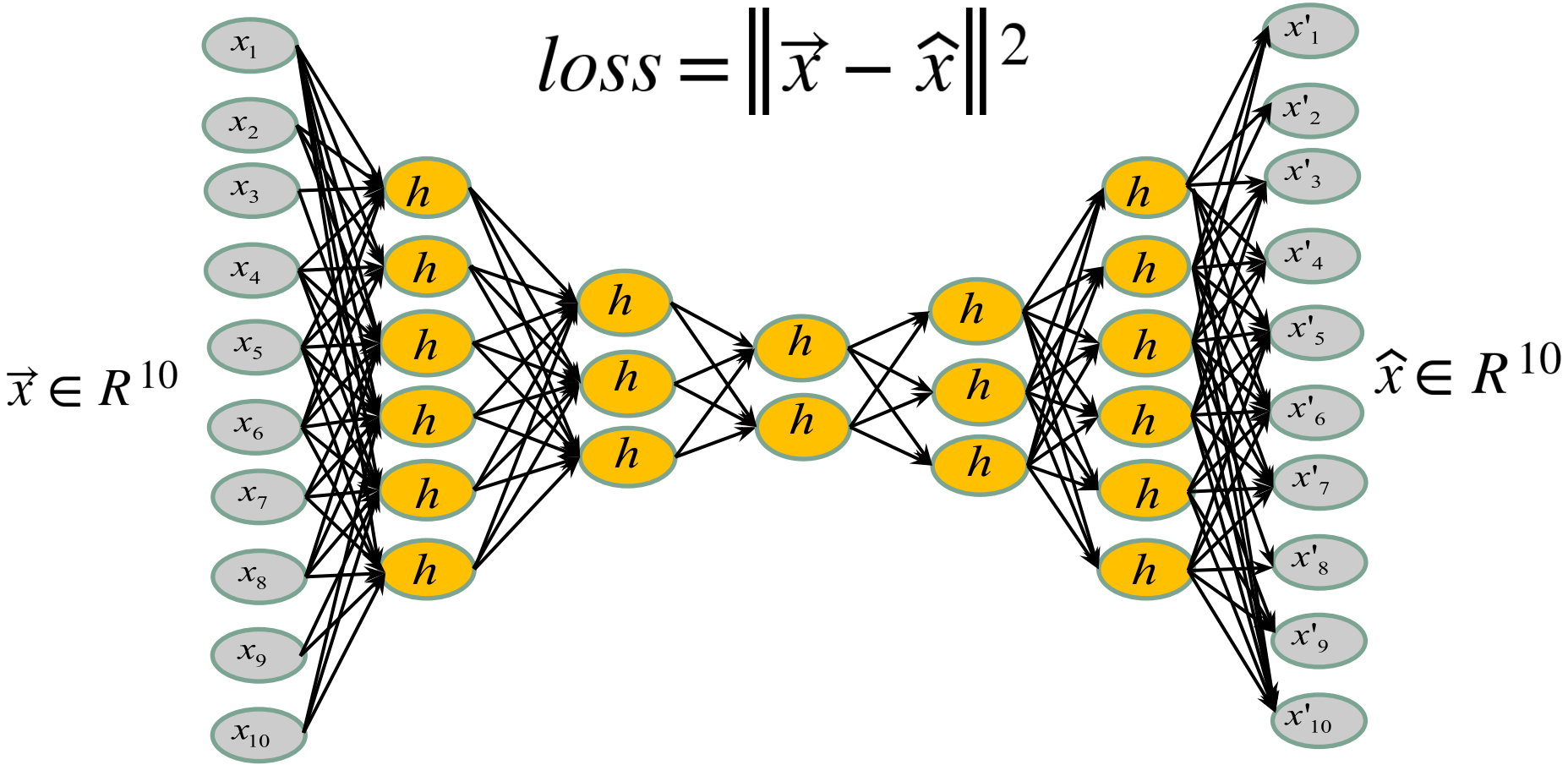
Summary

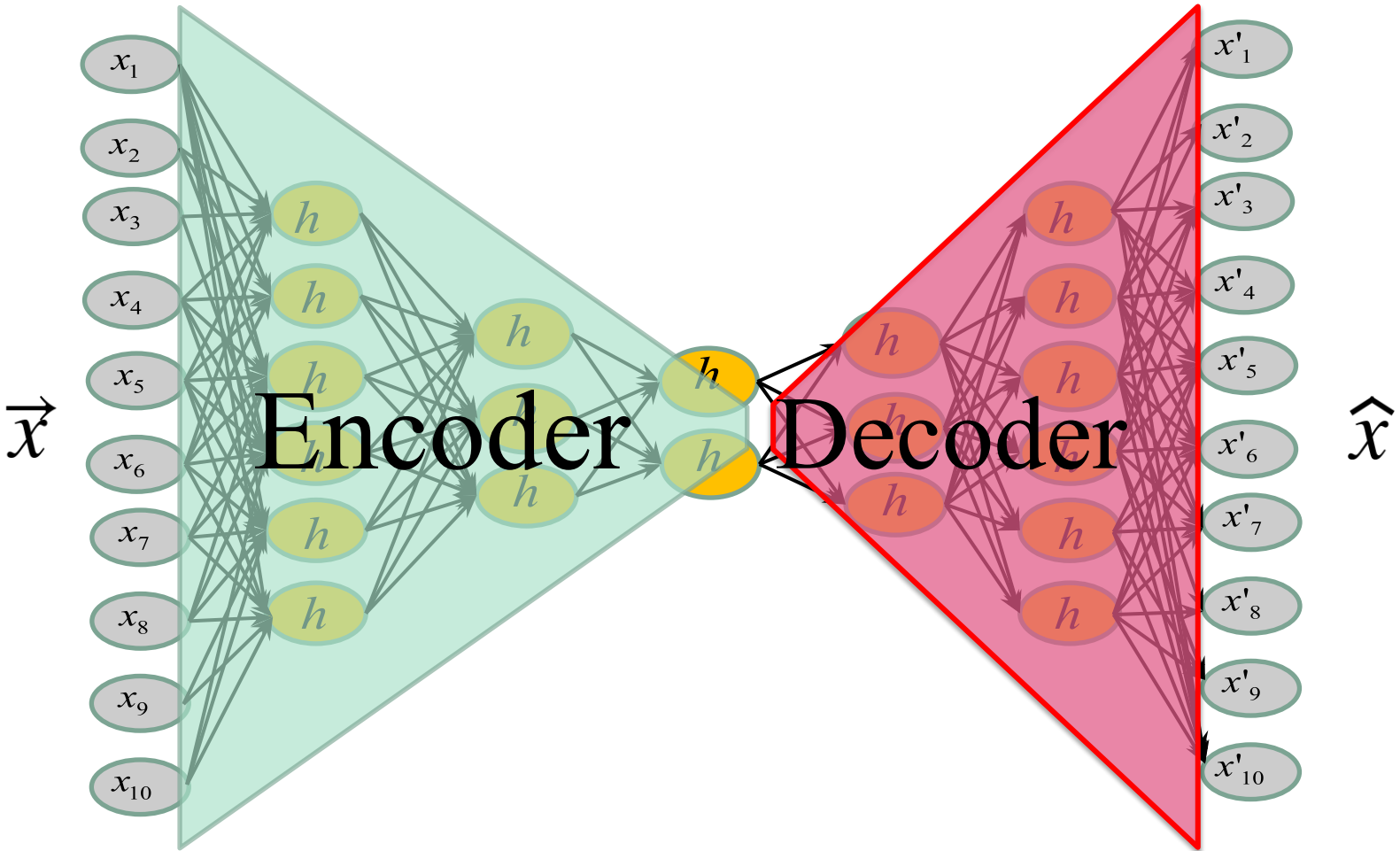
1. What are autoencoders and variational autoencoders?
2. How do they apply to MNIST (grayscale images)?
3. How do they apply to segmentation maps (ACDC cardiac labels)?

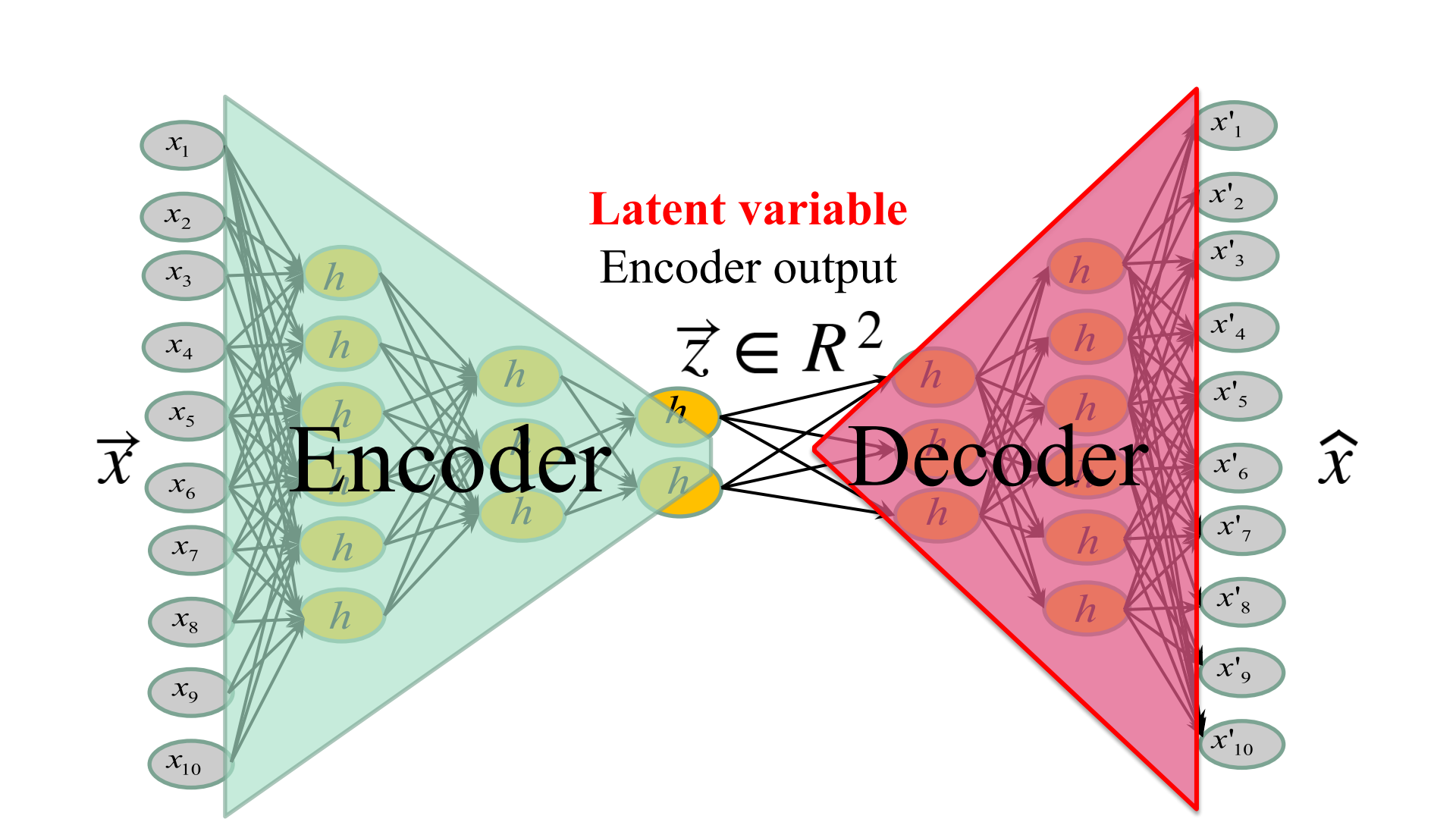
Goal : learn the latent representation of a set of data

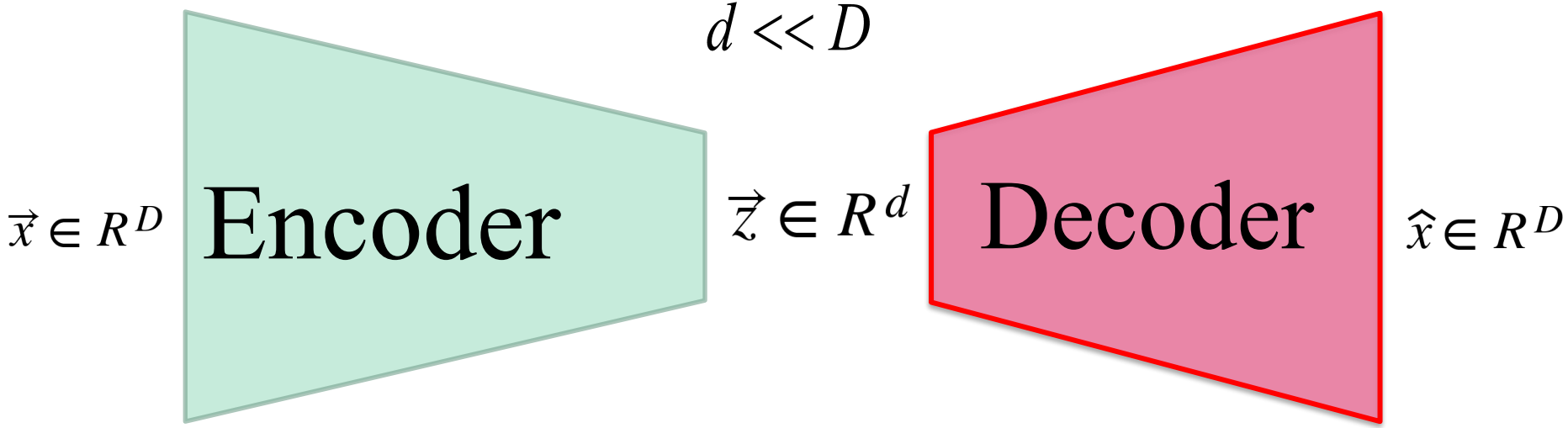
How : by training a Neural Net to output its own … input!

Note : if you are familiar with AE and VAE, you may skip the next couple of slides.


$loss = \|\vec{x} - \hat{x}\|^2$
$\vec{x} \in R^{10}$
$\hat{x} \in R^{10}$
x_1
x_2
x_3
x_4
x_5
x_6
x_7
x_8
x_9
x_{10}
x'_1
x'_2
x'_3
x'_4
x'_5
x'_6
x'_7
x'_8
x'_9
x'_{10}
h

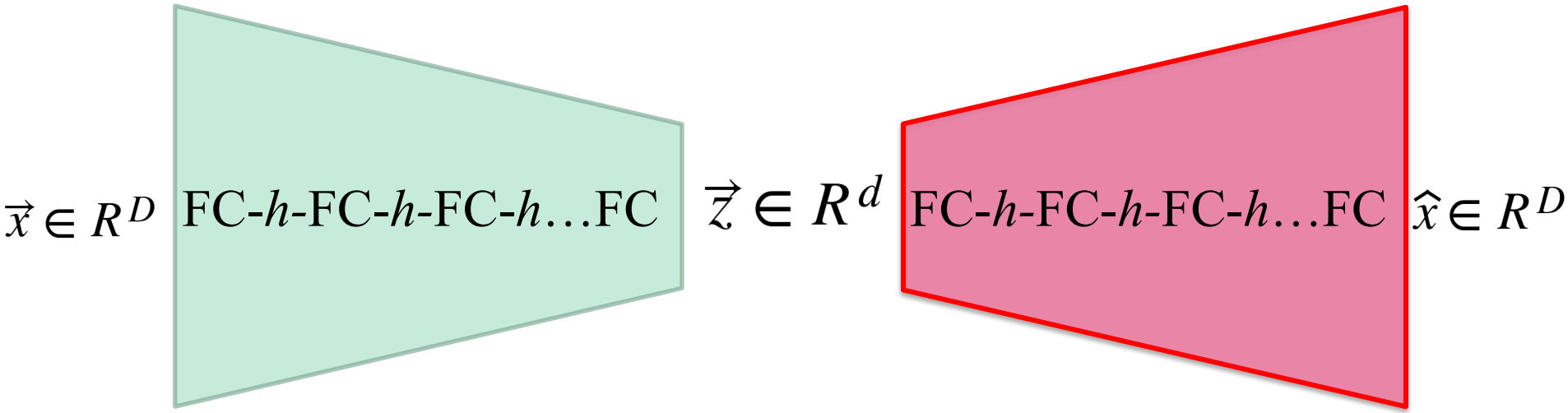

$\vec{x}$
x_1
x_2
x_3
x_4
x_5
x_6
x_7
x_8
x_9
x_{10}
Encoder
Decoder
$\hat{x}$
x'_1
x'_2
x'_3
x'_4
x'_5
x'_6
x'_7
x'_8
x'_9
x'_{10}


Latent variable
Encoder output
$\vec{z} \in R^2$
$\vec{x}$
Encoder
Decoder
$\hat{x}$
x_1
x_2
x_3
x_4
x_5
x_6
x_7
x_8
x_9
x_{10}
x'_1
x'_2
x'_3
x'_4
x'_5
x'_6
x'_7
x'_8
x'_9
x'_{10}
h

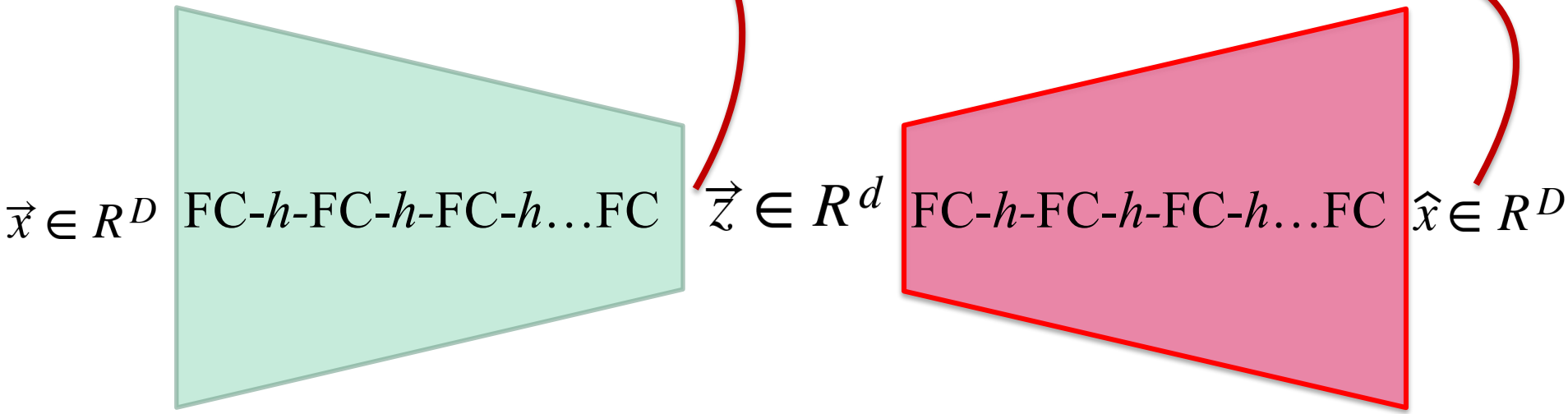

$d << D$
$\vec{x} \in R^D$
Encoder
$\vec{z} \in R^d$
Decoder
$\hat{x} \in R^D$

Fully-Connected Layers

h : activation function

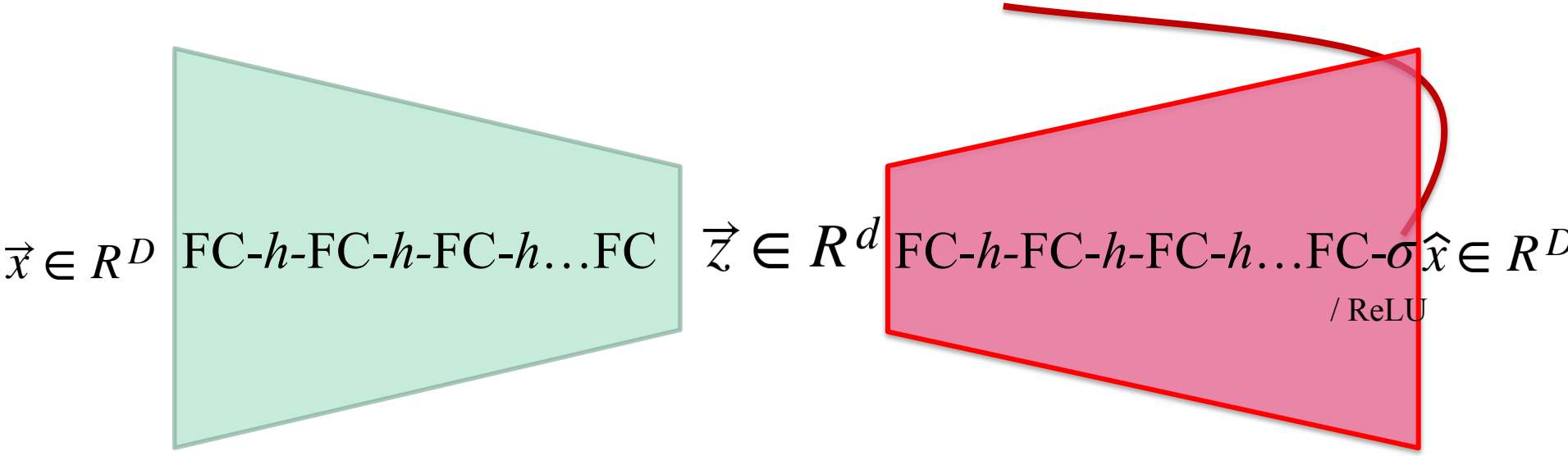


Very often...
No activation function at the output
of the encoder and the decoder



See **why?**

Sometimes **sigmoid** to predict pixel values between 0 and 1 or **ReLU** when the pixel values can be large but never negative.



The number of neurons

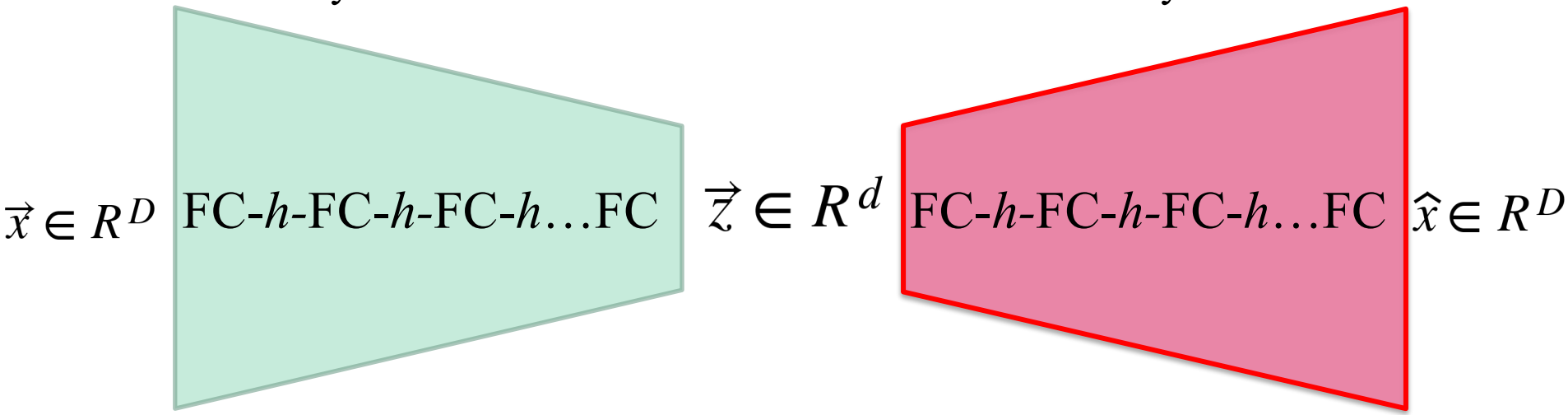
Decrease (or stay the same)

from a layer to another

The number of neurons

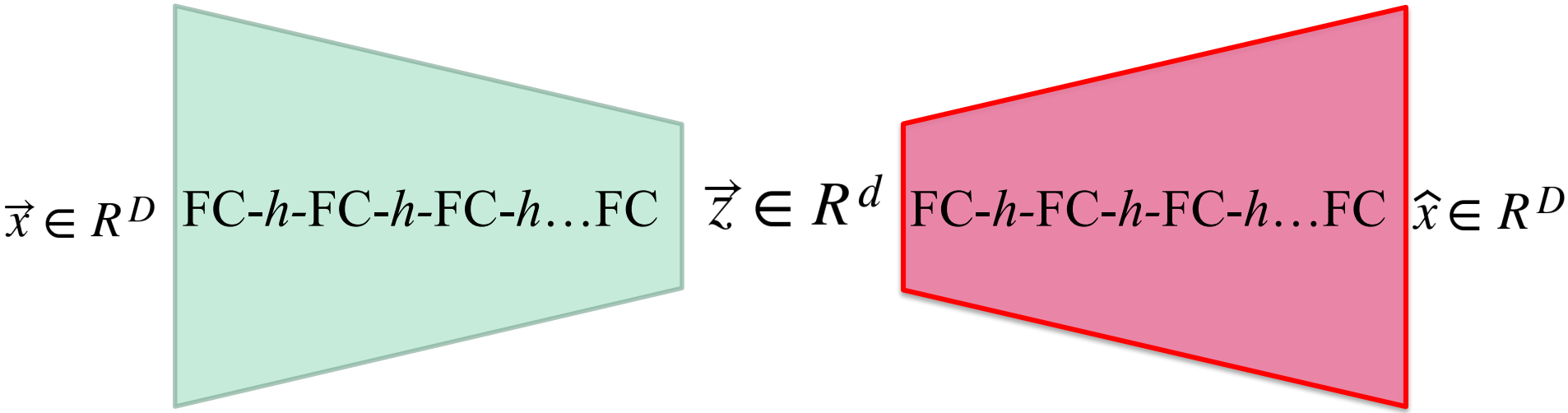
Increase (or stay the same)

from a layer to another

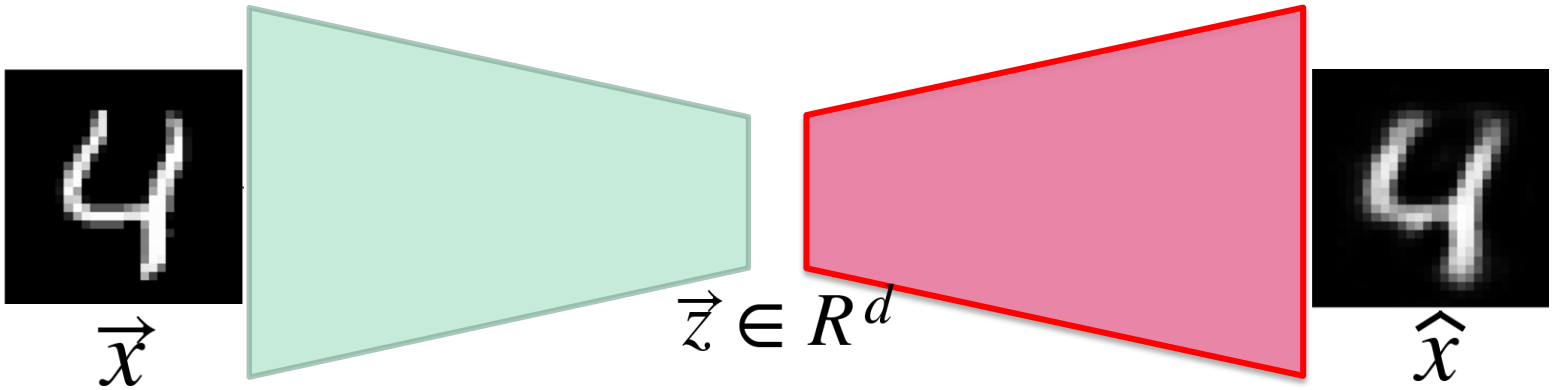


Very often...

The structure of the encoder is the **dual** of that of the decoder



Basic image-based autoencoder



Simple MNIST Autoencoder

```
class autoencoder(nn.Module):

    def __init__(self):

        super(autoencoder, self).__init__()

        self.encoder = nn.Sequential(

            nn.Linear(28 * 28, 128), nn.ReLU(True),

            nn.Linear(128, 64), nn.ReLU(True),

            nn.Linear(64, 12), nn.ReLU(True),

            nn.Linear(12, 2))

        self.decoder = nn.Sequential(

            nn.Linear(2, 12), nn.ReLU(True),

            nn.Linear(12, 64), nn.ReLU(True),

            nn.Linear(64, 128), nn.ReLU(True),

            nn.Linear(128, 28 * 28))


    def forward(self, x):

        z = self.encoder(x)

        x_prime = self.decoder(z)

        return x_prime
```

Latent space 2D

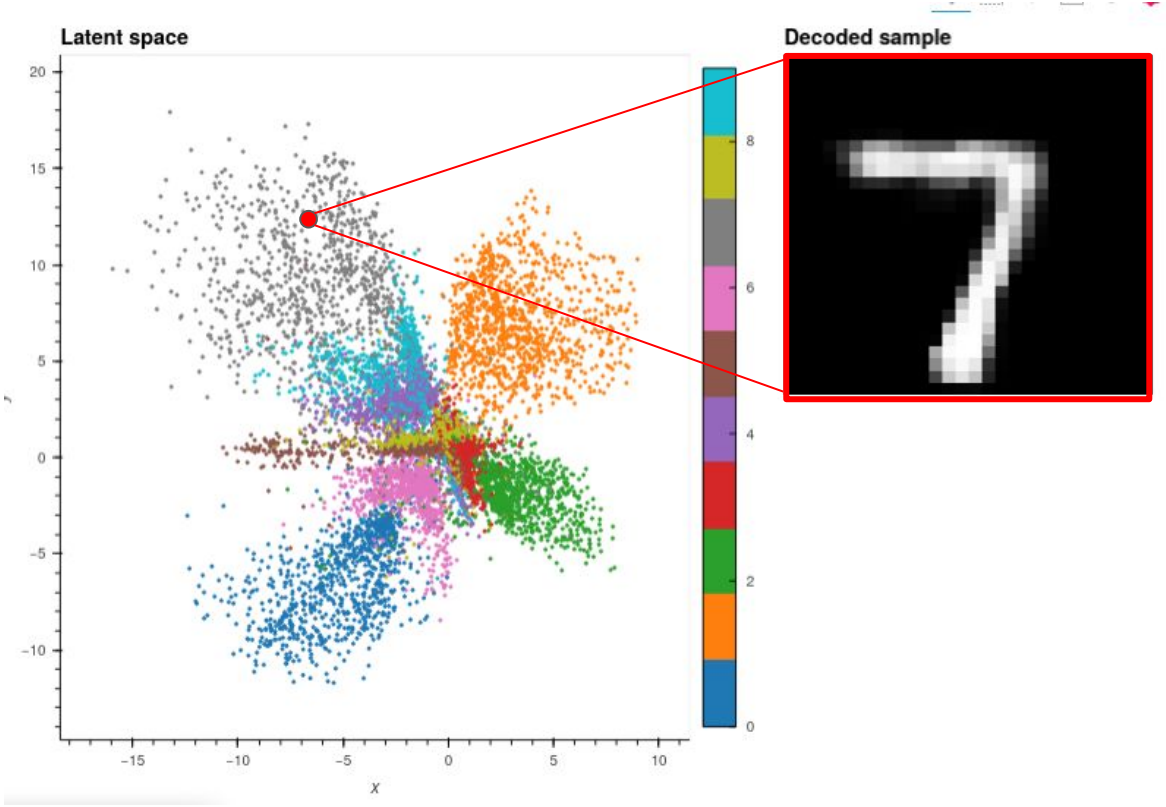
Simple MNIST Autoencoder

symmetry

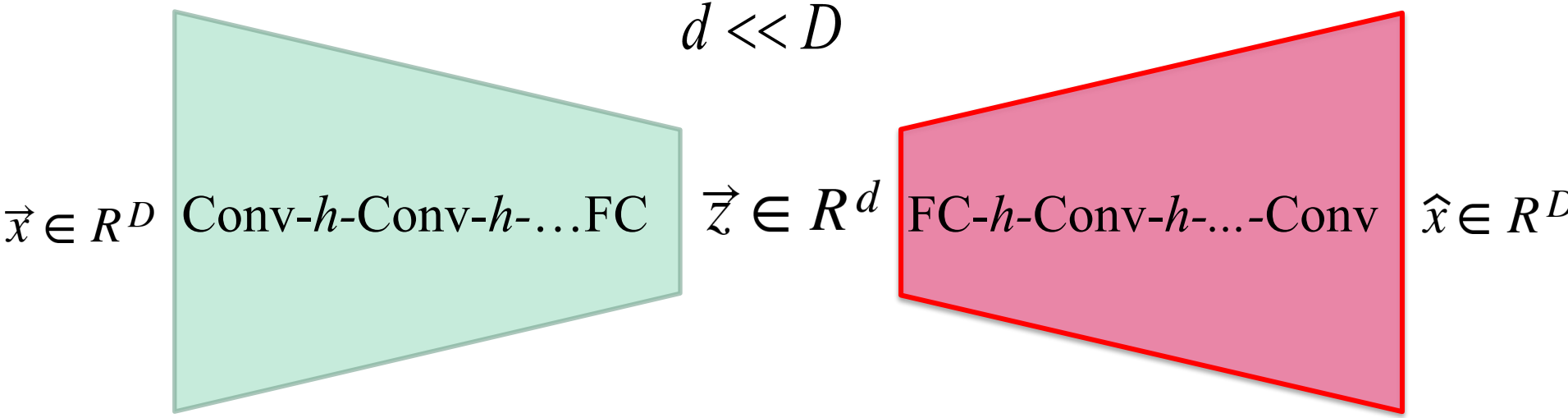
```
class autoencoder(nn.Module):

    def __init__(self):

        super(autoencoder, self).__init__()

        self.encoder = nn.Sequential(

            nn.Linear(28 * 28, 128), nn.ReLU(True),

            nn.Linear(128, 64), nn.ReLU(True),

            nn.Linear(64, 12), nn.ReLU(True),

            nn.Linear(12, 2))

        self.decoder = nn.Sequential(

            nn.Linear(2, 12), nn.ReLU(True),

            nn.Linear(12, 64), nn.ReLU(True),

            nn.Linear(64, 128), nn.ReLU(True),

            nn.Linear(128, 28 * 28))


    def forward(self, x):

        z = self.encoder(x)

        x_prime = self.decoder(z)

        return x_prime
```

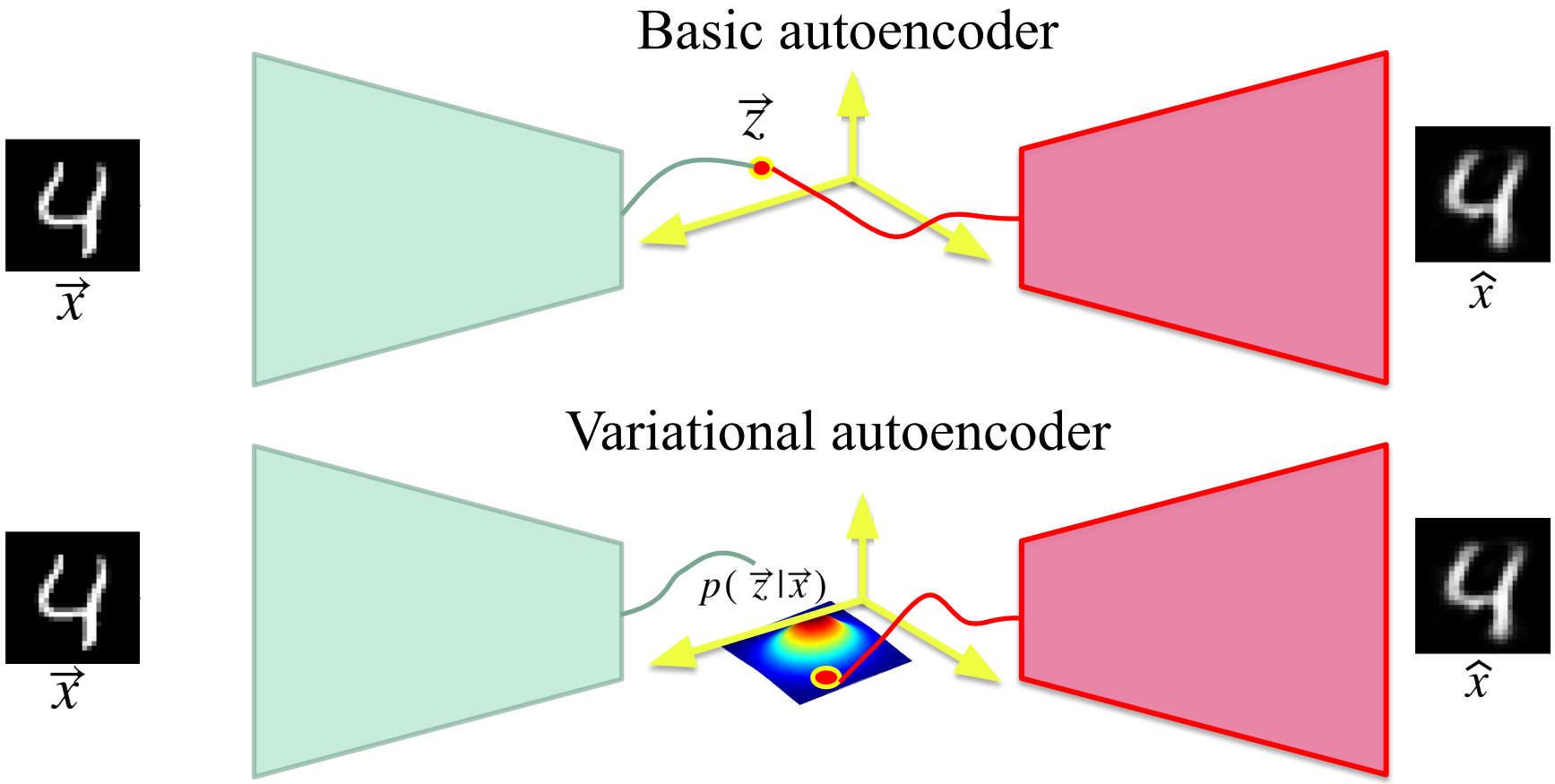
MNIST latent space (for 1000 images)

Each 2D point corresponds to an image

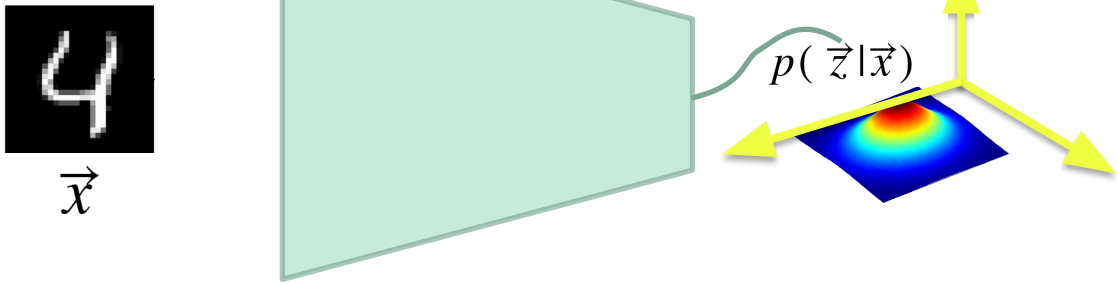


Conv layers


$d << D$
$\vec{x} \in R^D$
Conv-h-Conv-h-…FC
$\vec{z} \in R^d$
FC-h-Conv-h-...-Conv
$\hat{x} \in R^D$

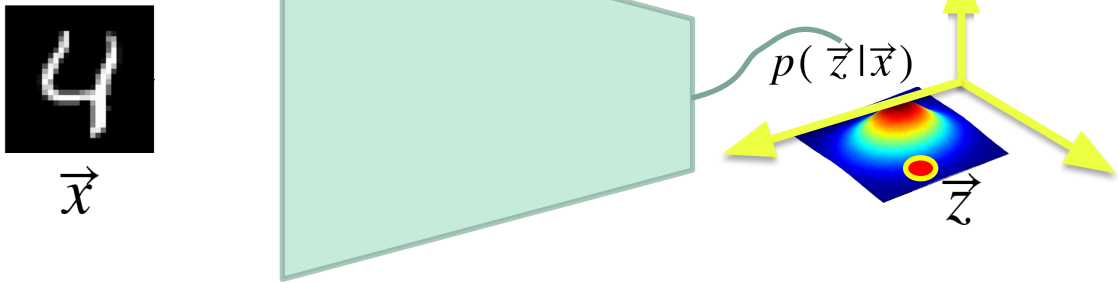

Basic autoencoder
$\vec{z}$
$\vec{x}$
$\hat{x}$
Variational autoencoder
$p(\vec{z}|\vec{x})$
$\vec{x}$
$\hat{x}$

Variational autoencoder



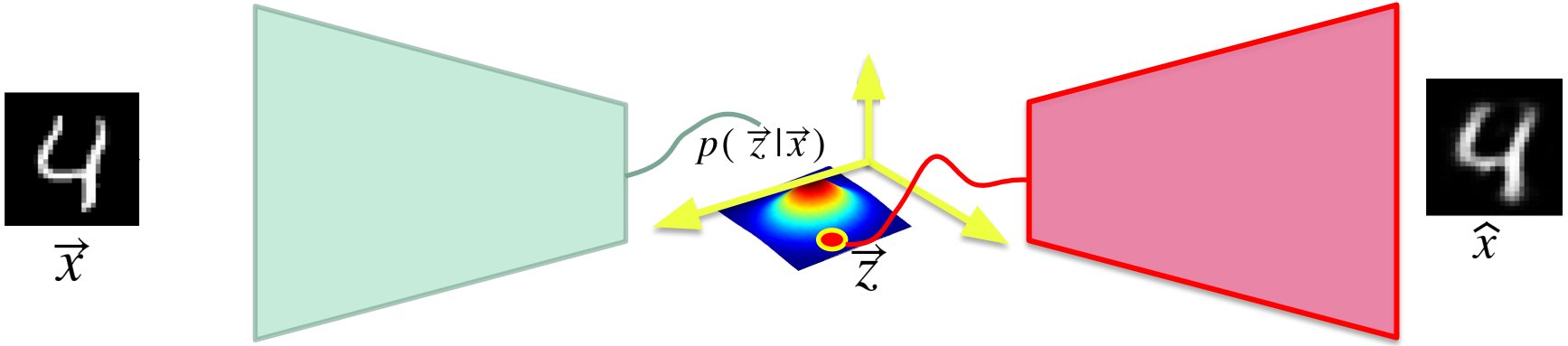
The encoder outputs a **distribution** $p(\vec{z}|\vec{x})$ and not just a **vector** $\vec{z}$

Variational autoencoder



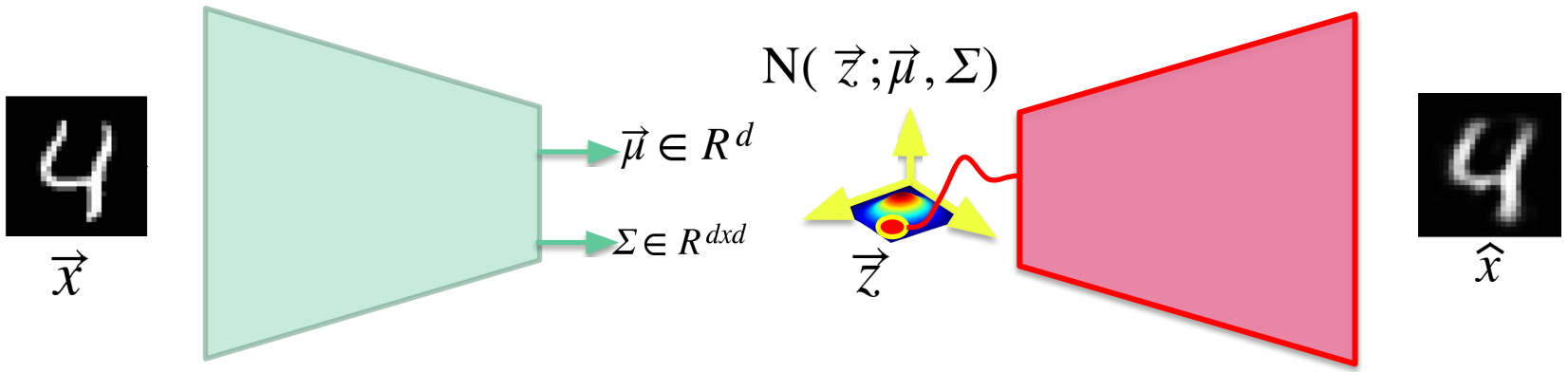
Random sample $\vec{z} \sim P(\vec{z}|\vec{x})$

Variational autoencoder

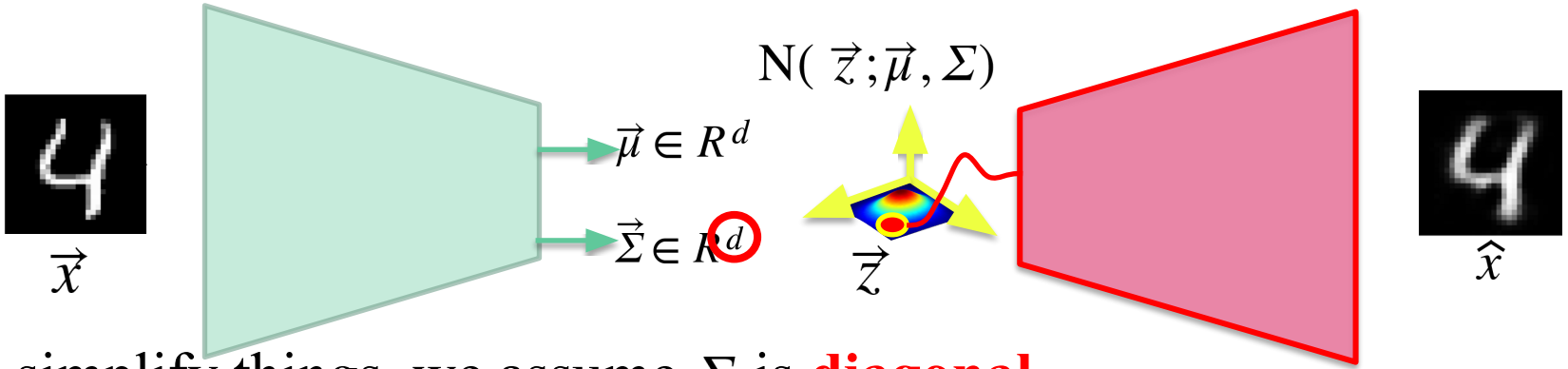


...and rebuild $\hat{x}$

$$p(\vec{z}|\vec{x}) \sim Gaussian$$



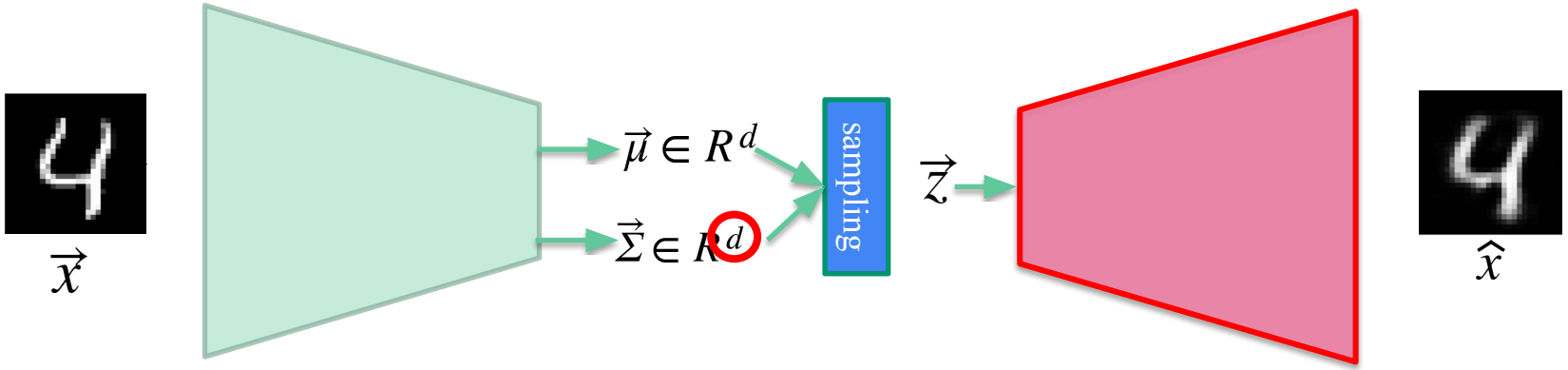
$$p(\vec{z}|\vec{x}) \sim Gaussian$$



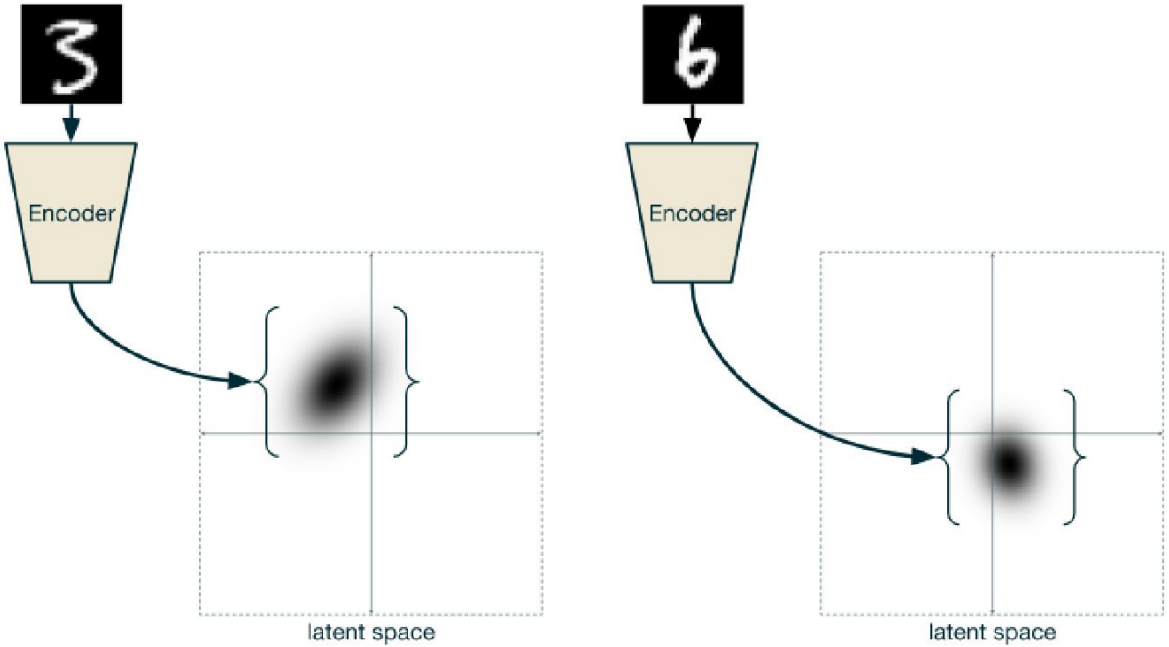
To simplify things, we assume Σ is **diagonal**

$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 & 0 & 0 \\ 0 & \sigma_1^2 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \sigma_d^2 \end{pmatrix}$$

Variational autoencoder

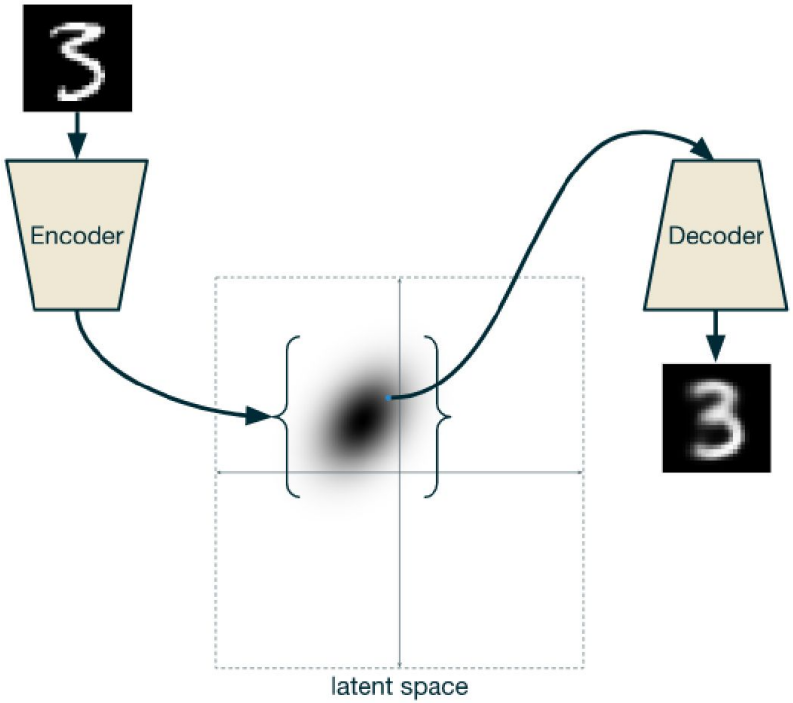


Another way of seeing things…



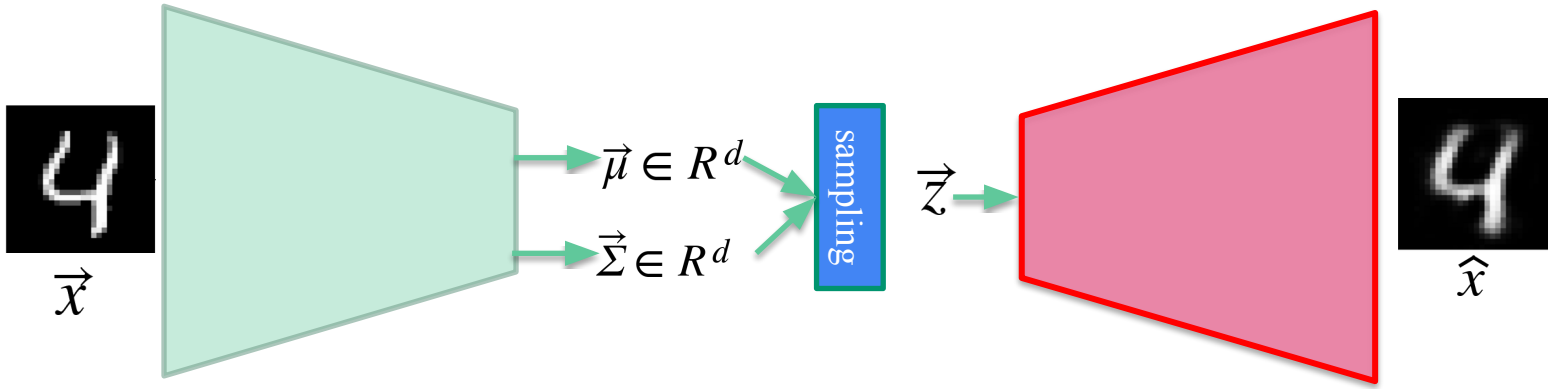
https://ijdykeman.github.io/ml/2016/12/21/cvae.html

Another way of seeing things…



https://ijdykeman.github.io/ml/2016/12/21/cvae.html

ELBO loss : Evidence Lower Bound loss

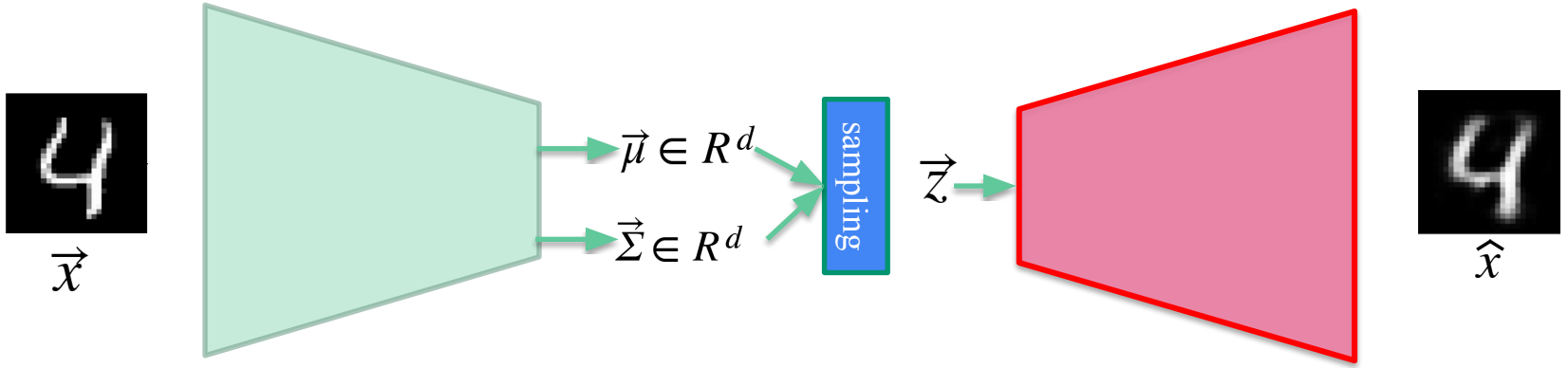


$$Loss = (\vec{x} - \hat{x})^2 + \lambda KL(N(\vec{z}; \vec{0}, \vec{1}), N(\vec{z}; \vec{\mu}, \vec{\Sigma}))$$

Loss decoder

Loss encoder

Other loss (in case the output is binary)

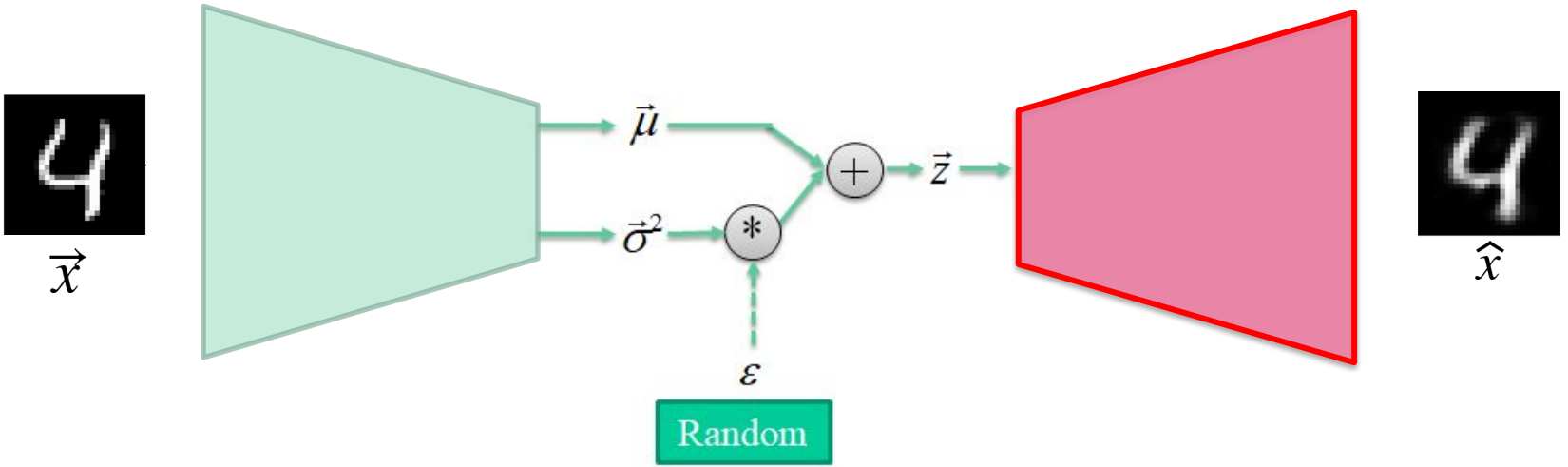


$$Loss = CrossEntropy(\vec{x}, \hat{x}) + \lambda KL\left(N(\vec{z}; \vec{0}, \vec{1}), N(\vec{z}; \vec{\mu}, \vec{\Sigma})\right)$$

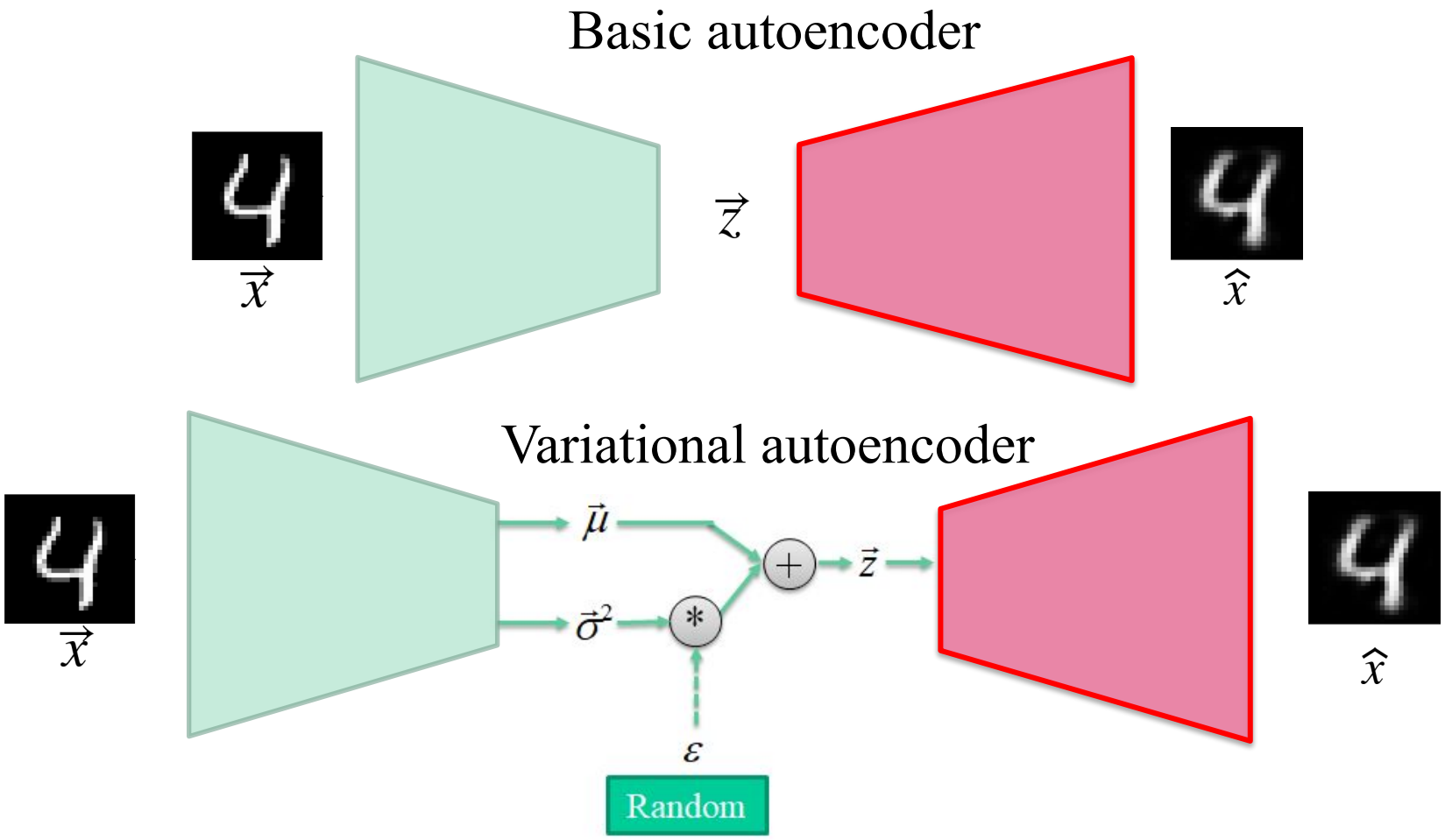
Loss decoder

Loss encoder

Reparametrization trick instead of sampling

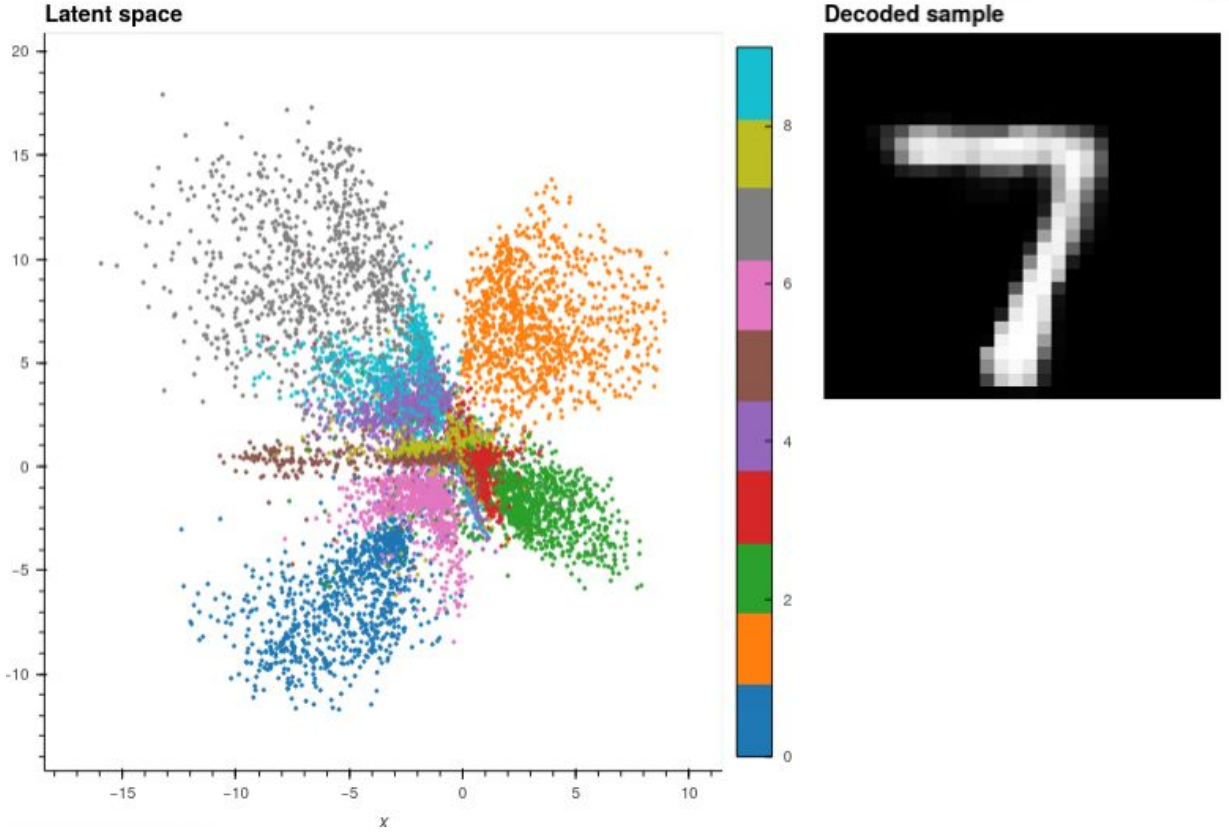


MNIST


Basic autoencoder
$\vec{x}$
$\vec{z}$
$\hat{x}$
Variational autoencoder
$\vec{x}$
$\vec{\mu}$
$\vec{\sigma}^2$
*
+
$\vec{z}$
$\hat{x}$
ε
Random

MNIST

Visualize the latent space (autoencoder)



MNIST

Visualize the latent space (variational autoencoder)